

P2609R0

Relaxing Ranges Just A Smidge

Published Proposal, 2022-06-20

This version:

<https://jehelset.gitlab.io/cpp/relaxing-ranges-just-a-smidge/P0.html>

Author:

[John Eivind Helset](#)

Audience:

SG9, LEWG

Toggle Diffs:

☐ Hide deleted text

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Subgroup:

SG9

Source:

<https://gitlab.com/jehelset/cpp/-/blob/main/sources/relaxing-ranges-just-a-smidge/P0.bs>

Table of Contents

1	Abstract
2	Problem
3	Proposal
4	Implementation
4.1	ranges-v3
4.2	libstdc++
5	Wording
5.1	23.2 Header <iterator> synopsis [iterator.synopsis]
5.2	23.3.2.4 Indirect callable traits [indirectcallable.traits]
5.3	23.3.6.4 Indirect callables [indirectcallable.indirectinvocable]
6	Acknowledgements
	References
	Informative References

§ 1. Abstract

Ranges algorithms that take a function and a projection should, in the unary case, constrain the function to enable:

```
iter_value_t<It> x = *it;  
f(proj(x));
```

Instead they are constrained to allow:

```
iter_value_t<projected<I,Proj>> u = proj(*it);  
f(u);
```

And likewise in the binary case. This is caused by the composition of indirect callable concepts with `projected`, seen for example in the constraints of `ranges::for_each` as `indirect_unary_invocable<projected<I,P>>`.

A fix is proposed that introduces a type-trait and makes a slight change to the definitions of the indirect callable concepts as well as `iter_common_reference_t`. The fix is a slight relaxation of the algorithmic constraints in ranges that does not break ABI.

§ 2. Problem

In [Niebler2015] Eric Niebler explains that the constraints put on the predicate of `ranges::unique_copy` must be strong enough to allow the algorithm to invoke it with a reference formed to a copy of an element:

Sometimes the algorithms call the predicates with references, sometimes with values, and sometimes — like `unique_copy` — with a mix of both.

That iteration of `ranges::unique_copy` did not have projections. The current iteration of `ranges::unique_copy` has projections, and is specified as:

```
template<input_range R, weakly_incrementable O, class Proj = identity,
        indirect_equivalence_relation<projected<iterator_t<R>, Proj>> C = ranges::equa
requires indirectly_copyable<iterator_t<R>, O> &&
        (forward_iterator<iterator_t<R>> ||
         (input_iterator<O> && same_as<range_value_t<R>, iter_value_t<O>>) ||
         indirectly_copyable_storable<iterator_t<R>, O>)
constexpr ranges::unique_copy_result<borrowed_iterator_t<R>, O>
        ranges::unique_copy(R&& r, O result, C comp = {}, Proj proj = {});
```

In [Revzin2022], Barry Revzin writes that projections are function adaptors, and do not change an algorithm.

Projections as function adaptors is also discussed in [ranges-v3/issues/233](#). In the unary case, projection is reduced to composition:

Put differently, every algorithm in the standard library that accepts a unary predicate and a projection, such that we have `algo(..., pred, proj)` can have its projection dropped without any loss of functionality because users can do the function composition themselves and invoke it as `algo(..., hof::compose(pred, proj))`.

Yet the constraints put on the triplet of iterator, function and projection fail to express projections as function adaptors. In the unary case they do not express `compose(f, proj)`. `ranges::for_each`, for example, takes a unary function `indirectly_unary_invocable<projected<I, Proj>> Fun`. The definitions of `indirectly_unary_invocable` and `projected` are:

```
template< class F, class I >
concept indirectly_unary_invocable =
    indirectly_readable<I> &&
    copy_constructible<F> &&
    invocable<F&, iter_value_t<I>&> &&
    invocable<F&, iter_reference_t<I>>> &&
    invocable<F&, iter_common_reference_t<I>>> &&
```

```
common_reference_with<
    invoke_result_t<F&, iter_value_t<I>&>,
    invoke_result_t<F&, iter_reference_t<I>>>>;
```

```
template<indirectly_readable I, indirectly_regular_unary_invocable<I> Proj>
struct projected {
    using value_type = remove_cvref_t<indirect_result_t<Proj&, I>>;
    indirect_result_t<Proj&, I> operator*() const; // not defined
};
```

Where `indirect_result_t` is:

```
template<class F, class... Is>
requires (indirectly_readable<Is> && ...) && invocable<F, iter_reference_t<Is>...>
using indirect_result_t = invoke_result_t<F, iter_reference_t<Is>...>;
```

The problematic constraint is `invocable<F &, iter_value_t<projected<I, Proj>> &>`, which becomes apparent when inlining the definition of `iter_value_t`:

```
invocable<F &, remove_cvref_t<invoke_result_t<Proj &, iter_reference_t<I>> &>
```

The code that this constraint is supposed to enable, and the code it actually enables, for copying and a referencing algorithm is:

Algorithm	Expected	Actual
Referencing	<pre>f(proj(*it))</pre>	<pre>f(proj(*it))</pre>
Copying	<pre>iter_value_t<It> v = *it; f(proj(v));</pre>	<pre>iter_value_t<projected<I,P>> u = proj(*it); f(u);</pre>

None of the algorithms in ranges are constrained to contain `f(u)`. They do not require a copyable or movable `u`. Requiring copyability or movability would not be enough. One would also need to somehow deal with the validity of `u`. There is no guarantee that the validity of `u` is not tied to the element `*it`. This was discussed in [ranges-v3/issues/148](#), and partially resolved by [\[P0740R0\]](#).

The actual consequence on user code seems small, as the malformed syntactic requirement of the constraint is often naturally covered. Valid, but perhaps rare, code will not compile or need work-arounds. In particular projections returning move-only types become problematic:

```
std::ranges::for_each(
    std::views::iota(0, 5),
    [](std::unique_ptr<int> v){ //Can't be invoked with std::unique_ptr<int> &
        std::cout << * v << std::endl;
```

```

},
[] (int v) {
    return std::make_unique<int>(v);
});

```

§ 3. Proposal

The proposal aims at a minimal fix. First a type-trait is defined that disambiguates between the indirect `value_type` of an iterator and a projection:

```

template<indirectly_readable I>
using indirect_value_t = see below;

```

The implementation technique is unspecified, but `indirect_value_t<I>` must be `iter_value_t<I>` & for an iterator and `invoke_result_t<Proj &, iter_value_t<Iter> &>` for `projected<Proj, Iter>`. The indirect callable concepts as well as `iter_common_reference_t` are now redefined to use `indirect_value_t` instead of `iter_value_t`, showcased here for `indirectly_unary_invocable`:

```

template< class F, class I >
concept indirectly_unary_invocable =
    indirectly_readable<I> &&
    copy_constructible<F> &&
    invocable<F&, indirect_value_t<I>iter_value_t<I>&> &&
    invocable<F&, iter_reference_t<I>>> &&
    invocable<F&, iter_common_reference_t<I>>> &&
    common_reference_with<
        invoke_result_t<F&, indirect_value_t<I>iter_value_t<I>&>,
        invoke_result_t<F&, iter_reference_t<I>>>>;

```

§ 4. Implementation

This proposal has currently been implemented and tested with both `ranges-v3` and `libstdc++` (on the gcc-12 release branch) without any issues.

§ 4.1. ranges-v3

`ranges-v3` has a firewalled `projected` (as proposed in [P2538R1]). Here the implementation uses a partial specialization that is constrained on the existence of a "secret" nested alias declaration. `ranges-v3` is a C++14

compatible library, and this is not a C++14 compatible implementation, but it the aim was to run the test-suite, not to learn new macros.

```
template<typename I>
struct indirect_value
{
    using type = iter_value_t<I> &;
};
template<typename I>
using indirect_value_t = typename indirect_value<I>::type;
```

```
template<typename I, typename Proj>
struct projected_
{
    struct type
    {
        using reference = indirect_result_t<Proj &, I>;
        using value_type = uncvref_t<reference>;
        reference operator*() const;

        using indirect_value_type = invoke_result_t<Proj &, iter_value_t<I> &>;
    };
};
```

```
template<typename T>
requires requires{ typename T::indirect_value_type; }
struct indirect_value<T>
{
    using type = typename T::indirect_value_type;
};
```

§ 4.2. libstdc++

libstdc++ has not yet firewalled `projected`, so here a simple partial specialization was used.

```
namespace __detail{

    template<typename _I>
    struct __indirect_value_impl
    { using type = iter_value_t<_I> &; };

} // namespace __detail
```

```
template<indirectly_readable _I>
    using indirect_value_t = typename __detail::__indirect_value_impl<_I>::type;
```

```
namespace __detail{

    template<weakly_incrementable _Iter, typename _Proj>
        struct __indirect_value_impl<projected<_Iter, _Proj>>
        { using type = invoke_result_t<_Proj &, iter_value_t<_Iter> &>; };

} // namespace __detail
```

§ 5. Wording

§ 5.1. 23.2 Header <iterator> synopsis [iterator.synopsis]

```
// [iterator.concepts], iterator concepts
// [iterator.concept.readable], concept indirectly_readable
template<class In>
    concept indirectly_readable = see below;

template<indirectly_readable T>
    using indirect_value_t = see below;

template<indirectly_readable T>
    using iter_common_reference_t =
        common_reference_t<iter_reference_t<T>, indirect_value_t<T>iter_value_t<T>&>;
```

§ 5.2. 23.3.2.4 Indirect callable traits [indirectcallable.traits]

1. To implement algorithms taking projections, it is necessary to determine the projected type of an iterators value type. Accordingly, it is required that the type `indirect_value_t<T>` denotes
 - `invoke_result_t<Proj&, iter_value_t<I> &>` if `T` names `projected<I, Proj>`
 - and `iter_value_t<T>` & otherwise.

§ 5.3. 23.3.6.4 Indirect callables [indirectcallable.indirectinvocable]

```
namespace std {
    template< class F, class I >
        concept indirectly_unary_invocable =
```

```

indirectly_readable<I> &&
copy_constructible<F> &&
invocable<F&, indirect_value_t<I>iter_value_t<I>&> &&
invocable<F&, iter_reference_t<I>> &&
invocable<F&, iter_common_reference_t<I>> &&
common_reference_with<
invoke_result_t<F&, indirect_value_t<I>iter_value_t<I>&>,
invoke_result_t<F&, iter_reference_t<I>>>;

```

```

template<class F, class I>
concept indirectly_regular_unary_invocable =
    indirectly_readable<I> &&
    copy_constructible<F> &&
    regular_invocable<F&, indirect_value_t<I>iter_value_t<I>&> &&
    regular_invocable<F&, iter_reference_t<I>> &&
    regular_invocable<F&, iter_common_reference_t<I>> &&
    common_reference_with<
invoke_result_t<F&, indirect_value_t<I>iter_value_t<I>&>,
invoke_result_t<F&, iter_reference_t<I>>>;

```

```

template<class F, class I>
concept indirect_unary_predicate =
    indirectly_readable<I> &&
    copy_constructible<F> &&
    predicate<F&, indirect_value_t<I>iter_value_t<I>&> &&
    predicate<F&, iter_reference_t<I>> &&
    predicate<F&, iter_common_reference_t<I>>;

```

```

template<class F, class I1, class I2>
concept indirect_binary_predicate =
    indirectly_readable<I1> && indirectly_readable<I2> &&
    copy_constructible<F> &&
    predicate<F&, indirect_value_t<I1>iter_value_t<I1>&, indirect_value_t<I1>iter_val
    predicate<F&, indirect_value_t<I1>iter_value_t<I1>&, iter_reference_t<I2>> &&
    predicate<F&, iter_reference_t<I1>, indirect_value_t<I1>iter_value_t<I2>&> &&
    predicate<F&, iter_reference_t<I1>, iter_reference_t<I2>> &&
    predicate<F&, iter_common_reference_t<I1>, iter_common_reference_t<I2>>;

```

```

template<class F, class I1, class I2 = I1>
concept indirect_equivalence_relation =
    indirectly_readable<I1> && indirectly_readable<I2> &&
    copy_constructible<F> &&
    equivalence_relation<F&, indirect_value_t<I>iter_value_t<I1>&, indirect_value_t<I
    equivalence_relation<F&, indirect_value_t<I>iter_value_t<I1>&, iter_reference_t<I
    equivalence_relation<F&, iter_reference_t<I1>, indirect_value_t<I>iter_value_t<I2
    equivalence_relation<F&, iter_reference_t<I1>, iter_reference_t<I2>> &&
    equivalence_relation<F&, iter_common_reference_t<I1>, iter_common_reference_t<I2>

```

```

template<class F, class I1, class I2 = I1>

```



```

concept indirect_strict_weak_order =
    indirectly_readable<I1> && indirectly_readable<I2> &&
    copy_constructible<F> &&
    strict_weak_order<F&, indirect_value_t<I1>iter_value_t<I1>&, indirect_value_t<I2>iter_value_t<I2>&>
    strict_weak_order<F&, indirect_value_t<I1>iter_value_t<I1>&, iter_reference_t<I2>>
    strict_weak_order<F&, iter_reference_t<I1>, indirect_value_t<I2>iter_value_t<I2>&>
    strict_weak_order<F&, iter_reference_t<I1>, iter_reference_t<I2>> &&
    strict_weak_order<F&, iter_common_reference_t<I1>, iter_common_reference_t<I2>>;

```

§ 6. Acknowledgements

The author would like to thank Barry Revzin, Christopher Di Bella, Arthur O'Dwyer and Hui Xie for guidance.

§ References

§ Informative References

[Niebler2015]

Eric Niebler. *Iterators++*, Part 3. URL: <https://ericniebler.com/2015/03/03/iterators-plus-plus-part-3/>

[P0740R0]

Casey Carter. *Ranges TS "Immediate" Issues from the July 2017 (Toronto) meeting*. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0740r0.html>

[P2538R1]

Arthur O'Dwyer; Casey Carter. *ADL-proof std::projected*. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2538r1.html>

[Revzin2022]

Barry Revzin. *Projections are Function Adaptors*. URL: <https://brevzin.github.io/c++/2022/02/13/projections-function-adaptors/>